Machine Learning Laboratory

Practical No.: 1

Feature Engineering for Machine Learning

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Apply basic feature engineering techniques such as handling missing values, encoding categorical variables, and scaling numerical attributes on a suitable open-access dataset to prepare a clean and model-ready dataset for machine learning analysis.

Program:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.impute import SimpleImputer


# Sample open-access style dataset (HR/Employee attrition style)
employee_data = {
    'Age': [25, 32, np.nan, 45, 29, 38, np.nan, 51],
    'Department': ['Sales', 'IT', 'HR', 'IT', 'Sales', np.nan, 'HR', 'IT'],
    'Salary': [40000, 55000, 48000, np.nan, 42000, 60000, 47000, 72000],
    'YearsExperience': [1, 5, 3, 12, 2, 8, 4, 15],
    'Attrition': ['Yes', 'No', 'No', 'No', 'Yes', 'No', 'Yes', 'No']
}
```

```
employee_df = pd.DataFrame(employee_data)
print("Original Dataset:\n", employee_df, "\n")
print("Missing values per column:\n", employee_df.isnull().sum(), "\n")

# Step 1: Handle missing values
numeric_imputer = SimpleImputer(strategy='mean')

employee_df['Age'] = numeric_imputer.fit_transform(employee_df[['Age']])
employee_df['Salary'] = numeric_imputer.fit_transform(employee_df[['Salary']])
categorical_imputer = SimpleImputer(strategy='most_frequent')

employee_df['Department'] =
categorical_imputer.fit_transform(employee_df[['Department']]).ravel()

print("After Handling Missing Values:\n", employee_df, "\n")

# Step 2: Encode categorical variables
label_encoder = LabelEncoder()

employee_df['Attrition_Encoded'] = label_encoder.fit_transform(employee_df['Attrition'])

encoded_df = pd.get_dummies(employee_df, columns=['Department'], prefix='Dept')

print("After One-Hot Encoding 'Department':\n", encoded_df, "\n")

# Step 3: Scale numerical attributes
standard_scaler = StandardScaler()
numerical_columns = ['Age', 'Salary', 'YearsExperience']

encoded_df[numerical_columns] =
standard_scaler.fit_transform(encoded_df[numerical_columns])
```

```
print("After Feature Scaling (Standardization):\n",
encoded_df[numerical_columns].round(3),"\n")


print("Final Model-Ready Dataset Shape:", encoded_df.shape)
```

Output:

Original Dataset:

	Age	Department	Salary	YearsExperience	Attrition
0	25.0	Sales	40000.0	1	Yes
1	32.0	IT	55000.0	5	No
2	NaN	HR	48000.0	3	No
3	45.0	IT	NaN	12	No
4	29.0	Sales	42000.0	2	Yes
5	38.0	NaN	60000.0	8	No
6	NaN	HR	47000.0	4	Yes
7	51.0	IT	72000.0	15	No

Missing values per column:

Age	2
Department	1
Salary	1
YearsExperience	0
Attrition	0
dtype:	int64

After Handling Missing Values:

	Age	Department	Salary	YearsExperience	Attrition
0	25.000000	Sales	40000.0	1	Yes
1	32.000000	IT	55000.0	5	No
2	36.666667	HR	48000.0	3	No
3	45.000000	IT	52000.0	12	No
4	29.000000	Sales	42000.0	2	Yes
5	38.000000	IT	60000.0	8	No
6	36.666667	HR	47000.0	4	Yes
7	51.000000	IT	72000.0	15	No

After One-Hot Encoding 'Department':

	Age	Salary	Years Experience	Attrition	Attrition_ Encoded	Dept_HR	Dept_IT	Dept_Sales
0	25.000000	40000.0	1	Yes	1	False	False	True
1	32.000000	55000.0	5	No	0	False	True	False
2	36.666667	48000.0	3	No	0	True	False	False
3	45.000000	52000.0	12	No	0	False	True	False
4	29.000000	42000.0	2	Yes	1	False	False	True
5	38.000000	60000.0	8	No	0	False	True	False
6	36.666667	47000.0	4	Yes	1	True	False	False
7	51.000000	72000.0	15	No	0	False	True	False

After Feature Scaling (Standardization):

	Age	Salary	YearsExperience
0	-1.486	-1.233	-1.121
1	-0.594	0.308	-0.267
2	0.000	-0.411	-0.694
3	1.064	0.000	1.228
4	-0.976	-1.027	-0.907
5	0.170	0.822	0.374
6	0.000	-0.514	-0.480
7	1.825	2.055	1.868

Final Model-Ready Dataset Shape: (8, 8)

Machine Learning Laboratory

Practical No.: 2

Dimensionality Reduction using PCA and LDA

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Make use of the dimensionality reduction process using PCA and LDA on a suitable open-access high-dimensional dataset and observe the influence of feature reduction on model performance and computational efficiency.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import time
```

```
wine_data = load_wine()
features, targets = wine_data.data, wine_data.target

print("Dataset shape:", features.shape, " Classes:", np.unique(targets))

scaled_features = StandardScaler().fit_transform(features)

features_train, features_test, targets_train, targets_test = train_test_split(
    scaled_features, targets, test_size=0.3, random_state=42
)

# Baseline (all 13 features)
start_time = time.time()

logistic_model = LogisticRegression(max_iter=1000).fit(
    features_train, targets_train
)

accuracy_full = accuracy_score(
    targets_test, logistic_model.predict(features_test)
)

time_full = time.time() - start_time

# PCA -> 2 components
pca_model = PCA(n_components=2)

features_train_pca = pca_model.fit_transform(features_train)
features_test_pca = pca_model.transform(features_test)
```

```
start_time = time.time()
logistic_model_pca = LogisticRegression(max_iter=1000).fit(
    features_train_pca, targets_train
)

accuracy_pca = accuracy_score(
    targets_test, logistic_model_pca.predict(features_test_pca)
)

time_pca = time.time() - start_time

print(
    f"\nPCA Explained Variance Ratio (2 components): "
    f"{pca_model.explained_variance_ratio_}"
)

print(
    f"Total variance retained: "
    f"{sum(pca_model.explained_variance_ratio_)*100:.2f}%"
)

# LDA -> 2 components (max = n_classes-1)
lda_model = LinearDiscriminantAnalysis(n_components=2)

features_train_lda = lda_model.fit_transform(
    features_train, targets_train
)

features_test_lda = lda_model.transform(features_test)
```

```
start_time = time.time()
logistic_model_lda = LogisticRegression(max_iter=1000).fit(
    features_train_lda, targets_train
)

accuracy_lda = accuracy_score(
    targets_test, logistic_model_lda.predict(features_test_lda)
)

time_lda = time.time() - start_time

print(
    f"\nAccuracy with all 13 features : "
    f"{accuracy_full:.4f} "
    f"(train time: {time_full*1000:.2f} ms)"
)

print(
    f"Accuracy with PCA (2 comp) : "
    f"{accuracy_pca:.4f} "
    f"(train time: {time_pca*1000:.2f} ms)"
)

print(
    f"Accuracy with LDA (2 comp) : "
    f"{accuracy_lda:.4f} "
    f"(train time: {time_lda*1000:.2f} ms)"
)

figure, axes = plt.subplots(1, 2, figsize=(11,4.5))
```

```
for class_id, plot_color, class_label in zip(
    [0,1,2],
    ['#4C72B0','#DD8452','#55A868'],
    wine_data.target_names
):
    axes[0].scatter(
        features_train_pca[targets_train==class_id,0],
        features_train_pca[targets_train==class_id,1],
        c=plot_color,
        label=class_label,
        s=25
    )

    axes[1].scatter(
        features_train_lda[targets_train==class_id,0],
        features_train_lda[targets_train==class_id,1],
        c=plot_color,
        label=class_label,
        s=25
    )

axes[0].set_title('PCA - Wine Dataset (2 Components)')
axes[0].set_xlabel('PC1')
axes[0].set_ylabel('PC2')
axes[0].legend()

axes[1].set_title('LDA - Wine Dataset (2 Components)')
axes[1].set_xlabel('LD1')
axes[1].set_ylabel('LD2')
axes[1].legend()
```

```
plt.tight_layout()
plt.savefig('imgs/a2_pca_lda.png', dpi=130)

print("\nPlot saved.")
```

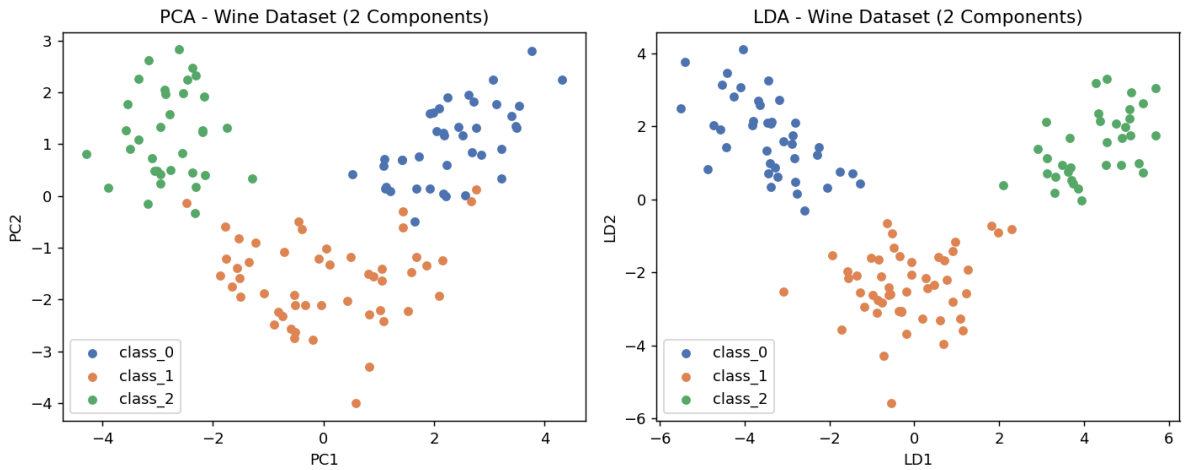
Output:

```
Dataset shape: (178, 13)  Classes: [0 1 2]

PCA Explained Variance Ratio (2 components): [0.36534816 0.18061255]
Total variance retained: 54.60%

Accuracy with all 13 features : 0.9815 (train time: 12.32 ms)
Accuracy with PCA (2 comp) : 0.9630 (train time: 3.64 ms)
Accuracy with LDA (2 comp) : 0.9815 (train time: 2.61 ms)

Plot saved.
```



Machine Learning Laboratory

Practical No.: 3

Multiple Linear Regression

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Develop a multiple linear regression model using a suitable real-world open-access dataset and assess its predictive capability using regression evaluation metrics such as MSE, RMSE, and R^2.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

diabetes_data = load_diabetes(as_frame=True)

features = diabetes_data.data
target = diabetes_data.target
```

```
print("Features:", list(features.columns))
print("Dataset shape:", features.shape)
print(features.head(), "\n")

features_train, features_test, target_train, target_test = train_test_split(
    features, target, test_size=0.2, random_state=42
)

regression_model = LinearRegression()
regression_model.fit(features_train, target_train)

predicted_values = regression_model.predict(features_test)

mean_squared_error_value = mean_squared_error(target_test, predicted_values)
root_mean_squared_error = np.sqrt(mean_squared_error_value)
r_squared = r2_score(target_test, predicted_values)

print("Model Coefficients:")

for feature_name, coefficient in zip(
    features.columns, regression_model.coef_
):
    print(f" {feature_name:>6}: {coefficient:8.3f}")

print(f"Intercept: {regression_model.intercept_:.3f}\n")

print(f"MSE: {mean_squared_error_value:.3f}")
print(f"RMSE: {root_mean_squared_error:.3f}")

print(f"R2: {r_squared:.4f}")
```

```
plt.figure(figsize=(6,5))

plt.scatter(
    target_test,
    predicted_values,
    alpha=0.7,
    color='#4C72B0',
    edgecolor='k'
)

plt.plot(
    [target.min(), target.max()],
    [target.min(), target.max()],
    'r--',
    lw=2
)

plt.xlabel('Actual Disease Progression')
plt.ylabel('Predicted Disease Progression')
plt.title('Multiple Linear Regression: Actual vs Predicted')

plt.tight_layout()
plt.savefig('imgs/a3_mlr.png', dpi=130)

print("\nPlot saved.")
```

Output:

Features: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']

Dataset shape: (442, 10)

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	0.017646
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641

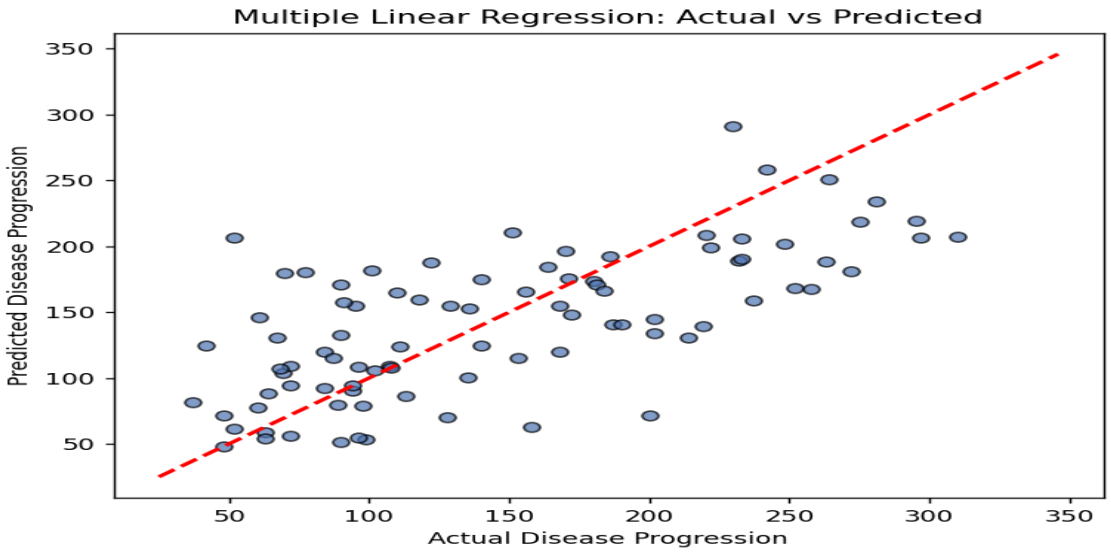
Model	Coefficients:
age:	37.904
sex:	-241.964
bmi:	542.429
bp:	347.704
s1:	-931.489
s2:	518.062
s3:	163.420
s4:	275.318
s5:	736.199
s6:	48.671
Intercept:	151.346

MSE: 2900.194

RMSE: 53.853

R2: 0.4526

Plot saved.



Machine Learning Laboratory

Practical No.: 4

Polynomial Regression

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Demonstrate the use of polynomial regression with varying polynomial degrees on a suitable open-access dataset and observe the effect of model complexity on prediction accuracy in terms of the bias–variance trade-off.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)

input_values = np.sort(np.random.rand(80, 1) * 10, axis=0)
output_values = (
```

```
    0.5 * input_values.ravel()**2
    - 2 * input_values.ravel()
    + 5
    + np.random.randn(80) * 3
)

input_train, input_test, output_train, output_test = train_test_split(
    input_values,
    output_values,
    test_size=0.2,
    random_state=42
)

polynomial_degrees = [1, 2, 4, 8]

plt.figure(figsize=(7,5))

plt.scatter(
    input_train,
    output_train,
    color='gray',
    s=20,
    label='Train data',
    alpha=0.6
)

input_plot = np.linspace(0, 10, 200).reshape(-1,1)

print(f"{'Degree':>7} | {'Train R2':>9} | {'Test R2':>8} | {'Test RMSE':>10}")
print("-"*45)
```

```
for degree_value in polynomial_degrees:

    polynomial_model = PolynomialFeatures(degree=degree_value)

    input_train_polynomial = polynomial_model.fit_transform(input_train)
    input_test_polynomial = polynomial_model.transform(input_test)

    regression_model = LinearRegression().fit(
        input_train_polynomial,
        output_train
    )

    train_r_squared = r2_score(
        output_train,
        regression_model.predict(input_train_polynomial)
    )

    predicted_test_values = regression_model.predict(input_test_polynomial)

    test_r_squared = r2_score(
        output_test,
        predicted_test_values
    )

    test_root_mean_squared_error = np.sqrt(
        mean_squared_error(output_test, predicted_test_values)
    )

    print(
        f"{degree_value:>7} | "
```

```
        f"{train_r_squared:>9.4f} | "
        f"{test_r_squared:>8.4f} | "
        f"{test_root_mean_squared_error:>10.3f}"
    )

    predicted_plot_values = regression_model.predict(
        polynomial_model.transform(input_plot)
    )

    plt.plot(
        input_plot,
        predicted_plot_values,
        label=f'Degree {degree_value}',
        linewidth=2
    )

plt.xlabel('X')
plt.ylabel('y')
plt.title('Polynomial Regression: Bias-Variance Trade-off')
plt.legend()
plt.tight_layout()

plt.savefig('imgs/a4_poly.png', dpi=130)

print(
    "\nObservation: Degree 1 underfits (high bias); "
    "Degree 8 starts overfitting (high variance,"
)

print(
```

"test R2 drops relative to degree 2-4 despite higher train R2)."

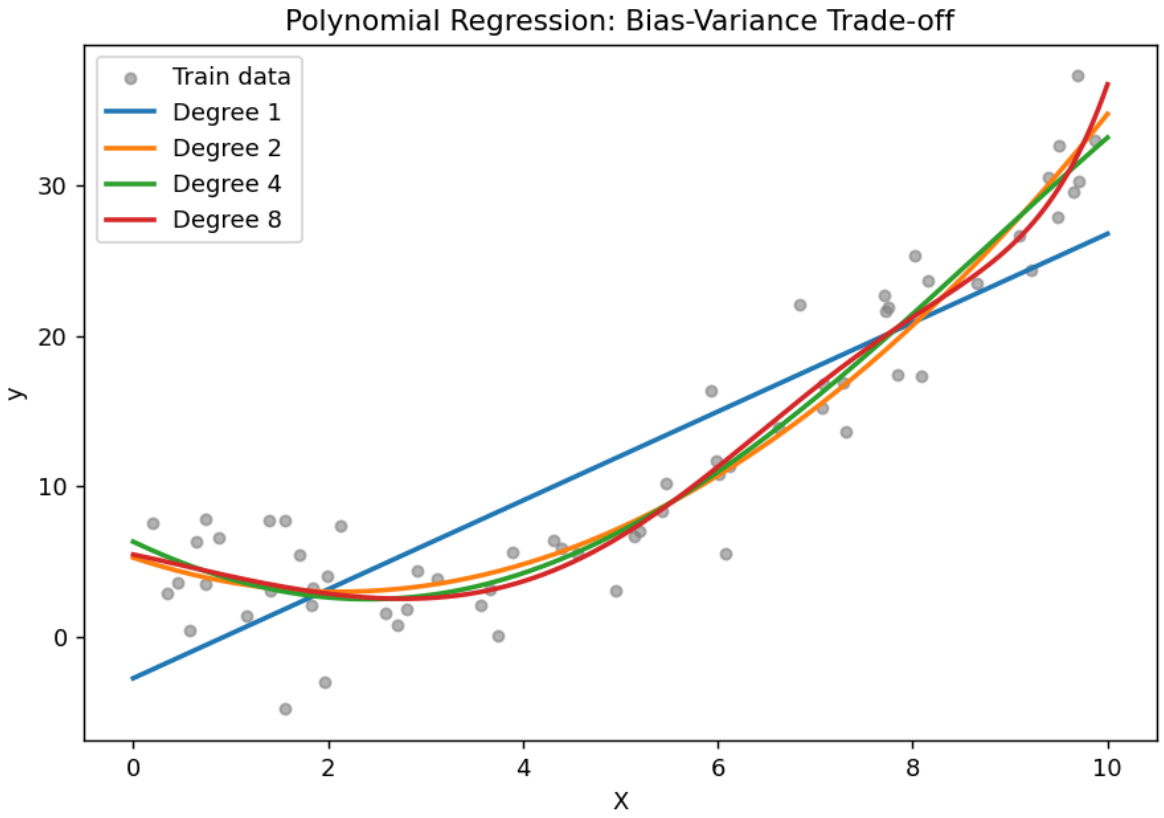
)

Output:

Degree | Train R2 | Test R2 | Test RMSE

1	0.7750	0.7611	4.222
2	0.9161	0.9179	2.475
4	0.9191	0.9122	2.560
8	0.9218	0.9053	2.658

Observation: Degree 1 underfits (high bias); Degree 8 starts overfitting (high variance, test R2 drops relative to degree 2-4 despite higher train R2).



Machine Learning Laboratory

Practical No.: 5

Regularization: Ridge and Lasso Regression

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Classify data using regularization techniques such as Ridge and Lasso regression on a suitable multivariate open-access dataset and observe the influence on model overfitting and multicollinearity.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

diabetes_data = load_diabetes(as_frame=True)

features, target = diabetes_data.data, diabetes_data.target
```

```
features_train, features_test, target_train, target_test = train_test_split(
    features,
    target,
    test_size=0.2,
    random_state=42
)

standard_scaler = StandardScaler()

features_train_scaled = standard_scaler.fit_transform(features_train)
features_test_scaled = standard_scaler.transform(features_test)

regression_models = {
    'Linear Regression': LinearRegression(),
    'Ridge (alpha=1.0)': Ridge(alpha=1.0),
    'Lasso (alpha=2.0)': Lasso(alpha=2.0)
}

print(
    f"{'Model':<20} | {'R2':>8} | {'RMSE':>8} | "
    f"{'Non-zero coefs':>15}"
)

print("-"*62)

model_results = {}

for model_name, regression_model in regression_models.items():

    regression_model.fit(features_train_scaled, target_train)
```

```
    predicted_values = regression_model.predict(features_test_scaled)

    r_squared = r2_score(target_test, predicted_values)

    root_mean_squared_error = np.sqrt(
        mean_squared_error(target_test, predicted_values)
    )

    non_zero_coefficients = np.sum(
        np.abs(regression_model.coef_) > 1e-4
    )

    model_results[model_name] = regression_model.coef_

    print(
        f"{model_name:<20} | "
        f"{r_squared:>8.4f} | "
        f"{root_mean_squared_error:>8.3f} | "
        f"{non_zero_coefficients:>15}"
    )

print("\nCoefficient comparison (feature shrinkage):")

print(
    f"{'Feature':<10}" +
    "".join(
        f"{model_name[:12]:>15}"
        for model_name in regression_models
    )
)
```

```
for feature_index, feature_name in enumerate(features.columns):

    coefficient_row = f"{feature_name:<10}"

    for model_name in regression_models:
        coefficient_row += (
            f"{model_results[model_name][feature_index]:>15.2f}"
        )

    print(coefficient_row)

figure, axes = plt.subplots(figsize=(9,5))

x_positions = np.arange(len(features.columns))
bar_width = 0.25

for model_index, (model_name, coefficients) in enumerate(
    model_results.items()
):

    axes.bar(
        x_positions + model_index * bar_width,
        coefficients,
        bar_width,
        label=model_name
    )

axes.set_xticks(x_positions + bar_width)
axes.set_xticklabels(features.columns)
axes.set_ylabel('Coefficient value')
```

```
axes.set_title(
    'Ridge vs Lasso: Coefficient Shrinkage (Diabetes Dataset)'
)
axes.legend()
axes.axhline(0, color='black', linewidth=0.8)

plt.tight_layout()

plt.savefig('imgs/a5_ridge_lasso.png', dpi=130)

print(
    "\nObservation: Lasso drives several coefficients to exactly zero "
    "(feature selection),"
)

print(
    "while Ridge shrinks all coefficients but keeps them non-zero, "
    "reducing multicollinearity impact."
)
```

Output:

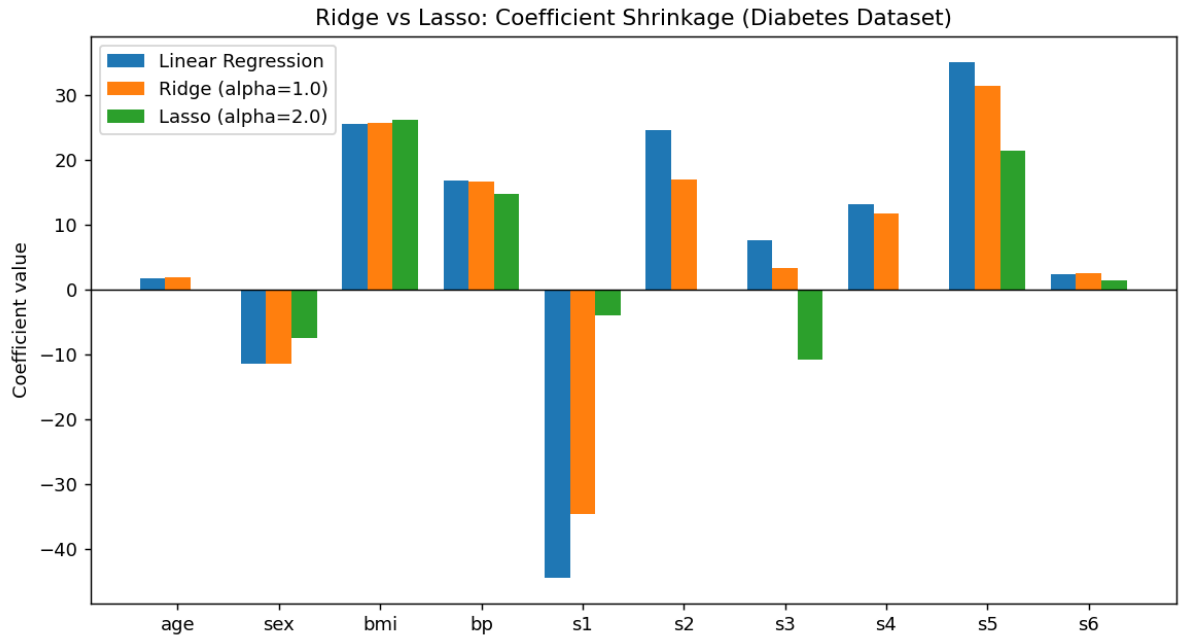
```
Model                |      R2    |    RMSE |  Non-zero coefs
-----------------------------------------------------------------------
Linear Regression    |  0.4526  |  53.853   |           10
Ridge (alpha=1.0)    |  0.4541  |  53.777   |           10
Lasso (alpha=2.0)    |  0.4717  |  52.904   |            7
```

Coefficient comparison (feature shrinkage):

Feature	Linear Regre	Ridge (alpha	Lasso (alpha
age	1.75	1.81	0.00
sex	-11.51	-11.45	-7.43
bmi	25.6	25.73	26.14
bp	16.83	16.73	14.74
s1	-44.45	-34.67	-4.07
s2	24.64	17.05	-0.00
s3	7.68	3.37	-10.87
s4	13.14	11.76	0.00
s5	35.16	31.38	21.50
s6	2.35	2.46	1.47

Observation: Lasso drives several coefficients to exactly zero (feature selection),

while Ridge shrinks all coefficients but keeps them non-zero, reducing multicollinearity impact.



Machine Learning Laboratory

Practical No.: 6

Logistic Regression for Binary Classification

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Illustrate the performance of a binary classification model using logistic regression on a suitable open-access dataset through a confusion matrix, precision, recall, and F1-score.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    accuracy_score,
```

```
    classification_report
)

cancer_data = load_breast_cancer()

features, target = cancer_data.data, cancer_data.target

print(
    "Dataset shape:", features.shape,
    " Classes:", cancer_data.target_names
)

features_train, features_test, target_train, target_test = train_test_split(
    features,
    target,
    test_size=0.25,
    random_state=42,
    stratify=target
)

standard_scaler = StandardScaler()

features_train_scaled = standard_scaler.fit_transform(features_train)
features_test_scaled = standard_scaler.transform(features_test)

logistic_model = LogisticRegression(max_iter=5000)

logistic_model.fit(features_train_scaled, target_train)

predicted_values = logistic_model.predict(features_test_scaled)
```

```
confusion_matrix_values = confusion_matrix(
    target_test,
    predicted_values
)

accuracy_value = accuracy_score(target_test, predicted_values)

precision_value = precision_score(
    target_test,
    predicted_values
)

recall_value = recall_score(
    target_test,
    predicted_values
)

f1_value = f1_score(
    target_test,
    predicted_values
)

print("\nConfusion Matrix:\n", confusion_matrix_values)

print(f"\nAccuracy: {accuracy_value:.4f}")
print(f"Precision: {precision_value:.4f}")
print(f"Recall: {recall_value:.4f}")
print(f"F1-score: {f1_value:.4f}")

print(
```

```
    "\nClassification Report:\n",
    classification_report(
        target_test,
        predicted_values,
        target_names=cancer_data.target_names
    )
)

figure, axes = plt.subplots(figsize=(5,4.5))

image = axes.imshow(
    confusion_matrix_values,
    cmap='Blues'
)

for row_index in range(2):
    for column_index in range(2):
        axes.text(
            column_index,
            row_index,
            confusion_matrix_values[row_index, column_index],
            ha='center',
            va='center',
            color='black',
            fontsize=14
        )

axes.set_xticks([0,1])
axes.set_yticks([0,1])
```

```
axes.set_xticklabels(cancer_data.target_names)
axes.set_yticklabels(cancer_data.target_names)

axes.set_xlabel('Predicted')
axes.set_ylabel('Actual')

axes.set_title(
    'Confusion Matrix - Logistic Regression\n'
    '(Breast Cancer Dataset)'
)

plt.colorbar(image)

plt.tight_layout()

plt.savefig(
    'imgs/a6_logreg_cm.png',
    dpi=130
)

print("\nPlot saved.")
```

Output:

```
Dataset shape: (569, 30)  Classes: ['malignant' 'benign']

Confusion Matrix:
 [[52  1]
 [ 1 89]]
```

Accuracy: 0.9860

Precision: 0.9889

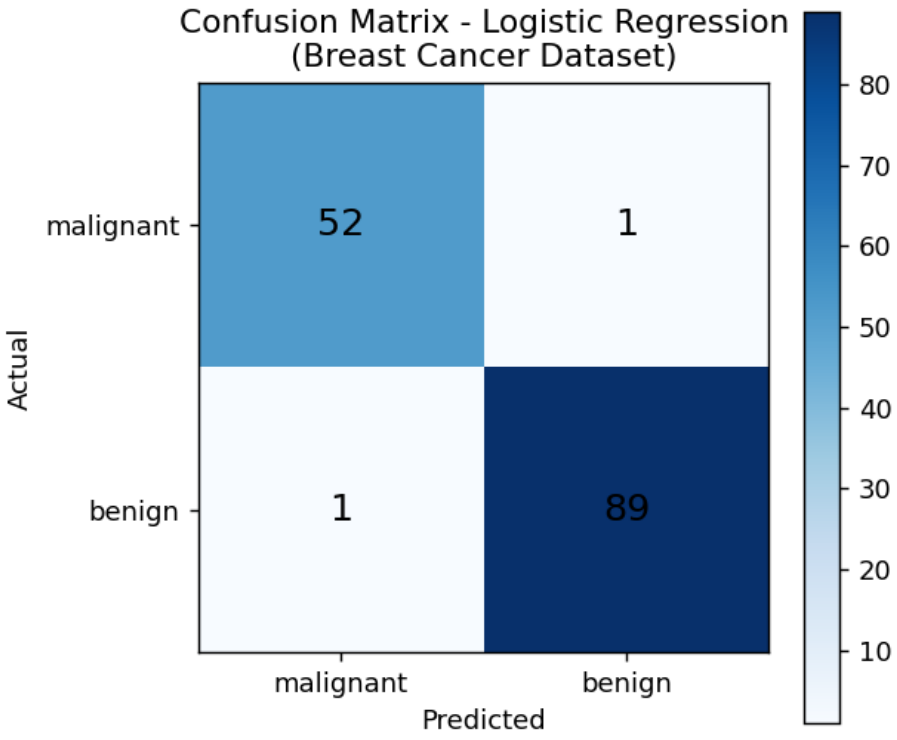
Recall: 0.9889

F1-score: 0.9889

Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	53
benign	0.99	0.99	0.99	90
accuracy			0.99	143
macro avg	0.99	0.99	0.99	143
weighted avg	0.99	0.99	0.99	143

Plot saved.



Machine Learning Laboratory

Practical No.: 7

Support Vector Machine Classifiers

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Utilize Support Vector Machine classifiers with linear and nonlinear kernels on a suitable open-access dataset and compare classification performance using appropriate evaluation metrics.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

iris_data = load_iris()

features = iris_data.data[:, [2,3]]  # petal length & width for 2D visualization
target = iris_data.target
```

```
features_train, features_test, target_train, target_test = train_test_split(
    features,
    target,
    test_size=0.25,
    random_state=42,
    stratify=target
)

standard_scaler = StandardScaler()

features_train_scaled = standard_scaler.fit_transform(features_train)
features_test_scaled = standard_scaler.transform(features_test)

kernel_types = ['linear', 'rbf', 'poly']

model_results = {}

print(f"{'Kernel':<10} | {'Accuracy':>9}")
print("-"*23)

for kernel_type in kernel_types:

    svm_model = SVC(
        kernel=kernel_type,
        C=1.0,
        gamma='scale'
    )

    svm_model.fit(features_train_scaled, target_train)
```

```
    accuracy_value = accuracy_score(
        target_test,
        svm_model.predict(features_test_scaled)
    )

    model_results[kernel_type] = (
        svm_model,
        accuracy_value
    )

    print(
        f"{kernel_type:<10} | {accuracy_value:>9.4f}"
    )

figure, axes = plt.subplots(
    1,
    3,
    figsize=(15,4.5)
)

grid_x, grid_y = np.meshgrid(
    np.linspace(
        features_train_scaled[:,0].min()-1,
        features_train_scaled[:,0].max()+1,
        200
    ),
    np.linspace(
        features_train_scaled[:,1].min()-1,
        features_train_scaled[:,1].max()+1,
        200
```

```
    )
)

for plot_axis, kernel_type in zip(axes, kernel_types):

    svm_model, accuracy_value = model_results[kernel_type]

    predicted_grid = svm_model.predict(
        np.c_[grid_x.ravel(), grid_y.ravel()]
    ).reshape(grid_x.shape)

    plot_axis.contourf(
        grid_x,
        grid_y,
        predicted_grid,
        alpha=0.3,
        cmap='viridis'
    )

    plot_axis.scatter(
        features_train_scaled[:,0],
        features_train_scaled[:,1],
        c=target_train,
        cmap='viridis',
        edgecolor='k',
        s=25
    )

    plot_axis.set_title(
        f'SVM ({kernel_type} kernel)\n'
```

```
        f'Accuracy: {accuracy_value:.2f}'
    )

    plot_axis.set_xlabel('Petal length (scaled)')
    plot_axis.set_ylabel('Petal width (scaled)')

plt.tight_layout()

plt.savefig(
    'imgs/a7_svm.png',
    dpi=130
)

print(
    "\nBest kernel:",
    max(model_results, key=lambda kernel_type: model_results[kernel_type][1])
)

print(
    "\nClassification Report (RBF kernel):\n",
    classification_report(
        target_test,
        model_results['rbf'][0].predict(features_test_scaled),
        target_names=iris_data.target_names
    )
)
```

Output:

Kernel | Accuracy

---

linear | 0.9211

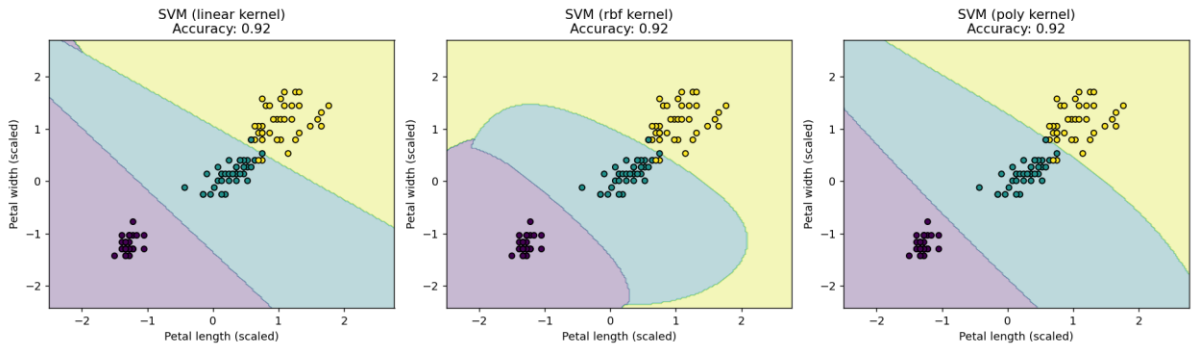
rbf | 0.9211

poly | 0.9211

Best kernel: linear

Classification Report (RBF kernel):

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	12
versicolor	0.86	0.92	0.89	13
virginica	0.92	0.85	0.88	13
accuracy			0.92	38
macro avg	0.92	0.92	0.92	38
weighted avg	0.92	0.92	0.92	38



Machine Learning Laboratory

Practical No.: 8

K-Means Clustering

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Apply the K-Means clustering algorithm on a suitable open-access dataset to obtain the optimal number of clusters using the Elbow or Silhouette method.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

iris_data = load_iris()

features = iris_data.data

scaled_features = StandardScaler().fit_transform(features)
```

```
wcss_values = []
silhouette_scores = []

cluster_range = range(2, 9)

for cluster_count in cluster_range:

    kmeans_model = KMeans(
        n_clusters=cluster_count,
        random_state=42,
        n_init=10
    )

    cluster_labels = kmeans_model.fit_predict(scaled_features)

    wcss_values.append(kmeans_model.inertia_)

    silhouette_scores.append(
        silhouette_score(scaled_features, cluster_labels)
    )

print(
    f"{'k':>3} | {'WCSS (Inertia)':>15} | "
    f"{'Silhouette Score':>17}"
)

print("-"*40)

for cluster_count, wcss_value, silhouette_value in zip(
    cluster_range,
```

```
    wcss_values,
    silhouette_scores
):
    print(
        f"{cluster_count:>3} | "
        f"{wcss_value:>15.3f} | "
        f"{silhouette_value:>17.4f}"
    )

optimal_clusters = list(cluster_range)[
    np.argmax(silhouette_scores)
]

print(
    f"\nOptimal k (highest silhouette score) = "
    f"{optimal_clusters}"
)

final_kmeans_model = KMeans(
    n_clusters=optimal_clusters,
    random_state=42,
    n_init=10
)

final_cluster_labels = final_kmeans_model.fit_predict(
    scaled_features
)

figure, axes = plt.subplots(
    1,
```

```
    3,
    figsize=(15,4.5)
)

axes[0].plot(
    list(cluster_range),
    wcss_values,
    'o-',
    color='#4C72B0'
)

axes[0].set_xlabel('Number of clusters (k)')
axes[0].set_ylabel('WCSS')
axes[0].set_title('Elbow Method')

axes[1].plot(
    list(cluster_range),
    silhouette_scores,
    'o-',
    color='#DD8452'
)

axes[1].set_xlabel('Number of clusters (k)')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('Silhouette Method')

scatter_plot = axes[2].scatter(
    features[:,2],
    features[:,3],
    c=final_cluster_labels,
```

```
    cmap='viridis',
    s=30,
    edgecolor='k'
)

axes[2].set_xlabel('Petal length')
axes[2].set_ylabel('Petal width')
axes[2].set_title(
    f'K-Means Clusters (k={optimal_clusters})'
)

plt.tight_layout()
plt.savefig(
    'imgs/a8_kmeans.png',
    dpi=130
)

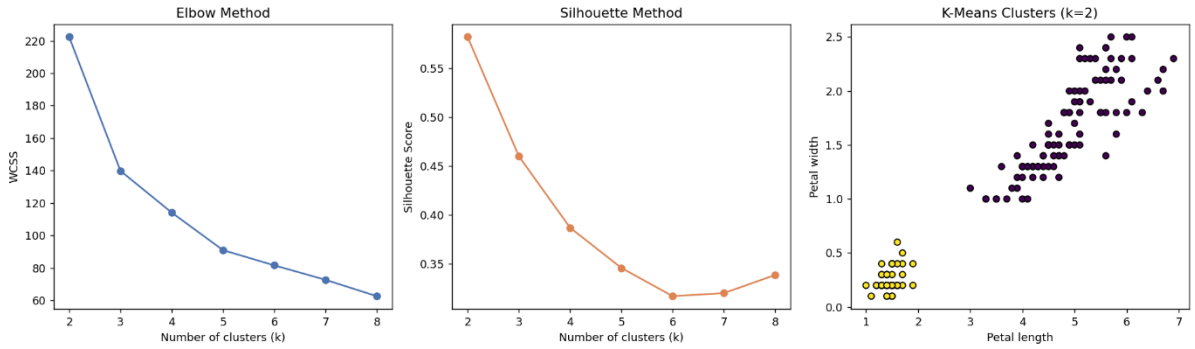
print("\nPlot saved.")
```

Output:

```
 k |  WCSS (Inertia) |  Silhouette Score
----------------------------------------
 2 |        222.362  |         0.5818
 3 |        139.820  |         0.4599
 4 |        114.093  |         0.3869
 5 |         90.928  |         0.3459
 6 |         81.544  |         0.3171
 7 |         72.631  |         0.3202
 8 |         62.541  |         0.3387
```

Optimal k (highest silhouette score) = 2

Plot saved.



Machine Learning Laboratory

Practical No.: 9

Density-Based Clustering using DBSCAN

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Employ a density-based clustering algorithm such as DBSCAN using a suitable open-access dataset to identify the clustering structure and noise points.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN, KMeans
from sklearn.metrics import silhouette_score

moon_features, true_labels = make_moons(
    n_samples=300,
    noise=0.08,
    random_state=42
)
```

```
scaled_features = StandardScaler().fit_transform(moon_features)

dbscan_model = DBSCAN(
    eps=0.25,
    min_samples=5
)

cluster_labels = dbscan_model.fit_predict(scaled_features)

cluster_count = len(set(cluster_labels)) - (
    1 if -1 in cluster_labels else 0
)

noise_count = list(cluster_labels).count(-1)

print(
    f"Estimated number of clusters: {cluster_count}"
)

print(
    f"Estimated number of noise points: {noise_count}"
)

if cluster_count > 1:

    silhouette_value = silhouette_score(
        scaled_features,
        cluster_labels
    )
```

```
    print(
        f"Silhouette Score: {silhouette_value:.4f}"
    )

kmeans_model = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=10
).fit(scaled_features)

figure, axes = plt.subplots(
    1,
    2,
    figsize=(11,4.5)
)

axes[0].scatter(
    moon_features[:,0],
    moon_features[:,1],
    c=kmeans_model.labels_,
    cmap='viridis',
    s=25,
    edgecolor='k'
)

axes[0].set_title(
    'K-Means (fails on non-convex shapes)'
)

axes[1].scatter(
```

```
    moon_features[:,0],
    moon_features[:,1],
    c=cluster_labels,
    cmap='viridis',
    s=25,
    edgecolor='k'
)

noise_mask = cluster_labels == -1

axes[1].scatter(
    moon_features[noise_mask,0],
    moon_features[noise_mask,1],
    c='red',
    marker='x',
    s=40,
    label='Noise'
)

axes[1].set_title(
    f'DBSCAN (clusters={cluster_count}, noise={noise_count})'
)

axes[1].legend()

plt.tight_layout()

plt.savefig(
    'imgs/a9_dbscan.png',
    dpi=130
```

```
)

print(
    "\nObservation: DBSCAN correctly separates the two crescent "
    "(non-convex) clusters and"
)

print(
    "flags outliers as noise, while K-Means "
    "(assumes spherical clusters) fails on this shape."
)
```

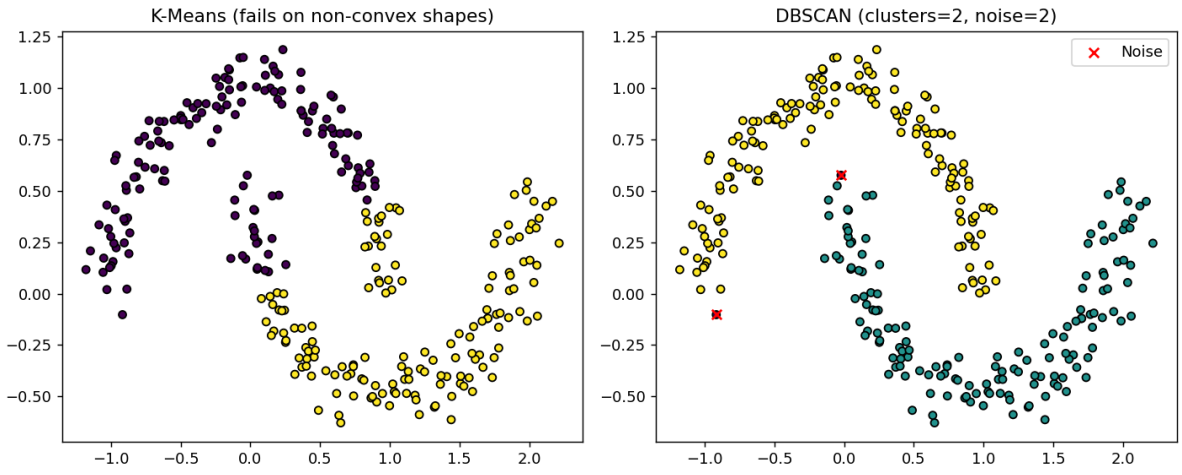
Output:

Estimated number of clusters: 2

Estimated number of noise points: 2

Silhouette Score: 0.1923

Observation: DBSCAN correctly separates the two crescent (non-convex) clusters and flags outliers as noise, while K-Means (assumes spherical clusters) fails on this shape.



Machine Learning Laboratory

Practical No.: 10

Ensemble Learning: Random Forest and Boosting

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Analyze the performance of ensemble learning techniques such as Random Forest or Boosting on a suitable open-access dataset in comparison with a baseline machine learning model.

Program:

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, f1_score

cancer_data = load_breast_cancer()

features, target = cancer_data.data, cancer_data.target
```

```
features_train, features_test, target_train, target_test = train_test_split(
    features,
    target,
    test_size=0.25,
    random_state=42,
    stratify=target
)

classification_models = {
    'Decision Tree (baseline)': DecisionTreeClassifier(random_state=42),
    'Random Forest (100 trees)': RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        random_state=42
    )
}

print(
    f"{'Model':<28} | {'Accuracy':>9} | {'F1-score':>9}"
)

print("-"*52)

accuracy_results = {}

for model_name, classification_model in classification_models.items():

    classification_model.fit(
```

```
        features_train,
        target_train
    )

    predicted_labels = classification_model.predict(
        features_test
    )

    accuracy_value = accuracy_score(
        target_test,
        predicted_labels
    )

    f1_value = f1_score(
        target_test,
        predicted_labels
    )

    accuracy_results[model_name] = accuracy_value

    print(
        f"{model_name:<28} | "
        f"{accuracy_value:>9.4f} | "
        f"{f1_value:>9.4f}"
    )

random_forest_model = classification_models[
    'Random Forest (100 trees)'
]
```

```
feature_importance_values = random_forest_model.feature_importances_

important_feature_indices = np.argsort(
    feature_importance_values
)[-10:]

figure, axes = plt.subplots(
    1,
    2,
    figsize=(12,5)
)

axes[0].barh(
    range(len(important_feature_indices)),
    feature_importance_values[important_feature_indices],
    color='#55A868'
)

axes[0].set_yticks(
    range(len(important_feature_indices))
)

axes[0].set_yticklabels(
    [
        cancer_data.feature_names[index]
        for index in important_feature_indices
    ]
)

axes[0].set_xlabel('Feature Importance')
```

```
axes[0].set_title(
    'Random Forest: Top 10 Important Features'
)

axes[1].bar(
    list(accuracy_results.keys()),
    list(accuracy_results.values()),
    color=['#C44E52','#4C72B0','#DD8452']
)

axes[1].set_ylabel('Accuracy')
axes[1].set_ylim(0.85,1.0)

axes[1].set_title(
    'Baseline vs Ensemble Comparison'
)

plt.setp(
    axes[1].get_xticklabels(),
    rotation=15,
    ha='right'
)

plt.tight_layout()

plt.savefig(
    'imgs/a10_ensemble.png',
    dpi=130
)
```

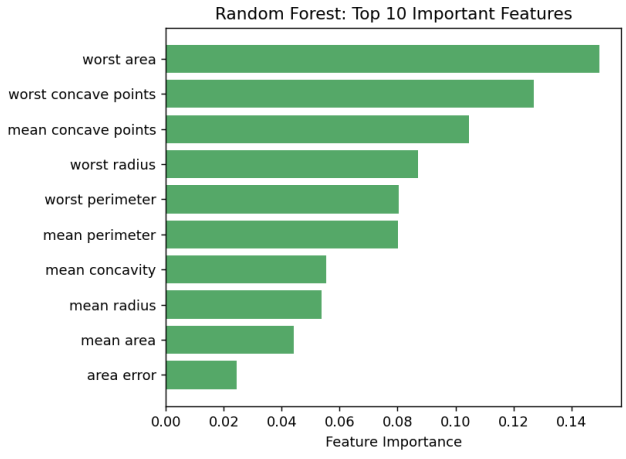
```
print(
    "\nObservation: Both ensemble methods "
    "(Random Forest, Gradient Boosting) outperform the"
)

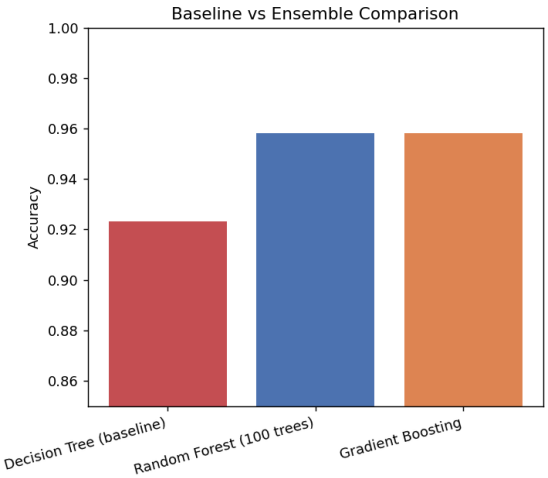
print(
    "single Decision Tree baseline by reducing variance "
    "(bagging) / bias (boosting) respectively."
)
```

Output:

Model	Accuracy	F1-score
Decision Tree (baseline)	0.9231	0.9379
Random Forest (100 trees)	0.9580	0.9670
Gradient Boosting	0.9580	0.9674

Observation: Both ensemble methods (Random Forest, Gradient Boosting) outperform the single Decision Tree baseline by reducing variance (bagging) / bias (boosting) respectively.





Machine Learning Laboratory

Practical No.: 11

Reinforcement Learning using Q-Learning

Name: Omkar Harish Shinde

TE(AI&DS) **Roll No.: 3164**

Problem Statement:

Evaluate the performance of reinforcement learning using the Q-Learning algorithm for a simple game or decision-making problem (Grid World), focusing on exploration–exploitation strategies and the optimal policy.

Program:

```
import numpy as np

import random

import matplotlib

matplotlib.use('Agg')

import matplotlib.pyplot as plt


# Simple Grid World: 4x4 grid, agent starts at (0,0), goal at (3,3), one obstacle at (1,1)


GRID_SIZE = 4

GOAL = (3, 3)

OBSTACLE = (1, 1)


ACTIONS = ['UP', 'DOWN', 'LEFT', 'RIGHT']
```

```
ACTION_EFFECTS = {
    'UP': (-1,0),
    'DOWN': (1,0),
    'LEFT': (0,-1),
    'RIGHT': (0,1)
}

q_table = np.zeros(
    (GRID_SIZE, GRID_SIZE, len(ACTIONS))
)

learning_rate = 0.1
discount_factor = 0.9
exploration_rate = 0.2
total_episodes = 500

def take_step(current_state, action_index):

    row_change, column_change = ACTION_EFFECTS[
        ACTIONS[action_index]
    ]

    current_row, current_column = current_state

    next_row = max(
        0,
        min(
            GRID_SIZE-1,
            current_row + row_change
        )
```

```
    )

    next_column = max(
        0,
        min(
            GRID_SIZE-1,
            current_column + column_change
        )
    )

    if (next_row, next_column) == OBSTACLE:
        next_row, next_column = current_row, current_column

    step_reward = (
        10
        if (next_row, next_column) == GOAL
        else -1
    )

    episode_done = (
        (next_row, next_column) == GOAL
    )

    return (
        (next_row, next_column),
        step_reward,
        episode_done
    )

episode_rewards = []
```

```
for episode_number in range(total_episodes):

    current_state = (0, 0)
    accumulated_reward = 0

    for time_step in range(100):

        if random.random() < exploration_rate:
            selected_action = random.randint(0, 3)
        else:
            selected_action = np.argmax(
                q_table[
                    current_state[0],
                    current_state[1]
                ]
            )

        next_state, step_reward, episode_done = take_step(
            current_state,
            selected_action
        )

        best_next_value = np.max(
            q_table[
                next_state[0],
                next_state[1]
            ]
        )

        q_table[
```

```
            current_state[0],
            current_state[1],
            selected_action
        ] += learning_rate * (
            step_reward
            + discount_factor * best_next_value
            - q_table[
                current_state[0],
                current_state[1],
                selected_action
            ]
        )

        current_state = next_state
        accumulated_reward += step_reward

        if episode_done:
            break

    episode_rewards.append(accumulated_reward)

    exploration_rate = max(
        0.01,
        exploration_rate * 0.995
    )

print(
    "Training complete over",
    total_episodes,
    "episodes."
```

```
)

print(
    f"Average reward (first 50 episodes) : "
    f"{np.mean(episode_rewards[:50]):.2f}"
)

print(
    f"Average reward (last 50 episodes) : "
    f"{np.mean(episode_rewards[-50:]):.2f}"
)

# Derive optimal policy
print("\nLearned Optimal Policy (grid view):")

policy_symbols = {
    'UP':'^',
    'DOWN':'v',
    'LEFT':'<',
    'RIGHT':'>'
}

for grid_row in range(GRID_SIZE):

    row_string = ""

    for grid_column in range(GRID_SIZE):

        if (grid_row, grid_column) == GOAL:
            row_string += " G "
```

```
        elif (grid_row, grid_column) == OBSTACLE:
            row_string += " X "

        else:
            best_action = ACTIONS[
                np.argmax(
                    q_table[
                        grid_row,
                        grid_column
                    ]
                )
            ]

            row_string += (
                f" {policy_symbols[best_action]} "
            )

    print(row_string)

plt.figure(figsize=(7,4.5))

moving_window = 20

smoothed_rewards = np.convolve(
    episode_rewards,
    np.ones(moving_window) / moving_window,
    mode='valid'
)

plt.plot(
```

```
    episode_rewards,
    alpha=0.3,
    color='gray',
    label='Reward per episode'
)

plt.plot(
    range(
        moving_window-1,
        len(episode_rewards)
    ),
    smoothed_rewards,
    color='#4C72B0',
    linewidth=2,
    label=f'{moving_window}-episode moving avg'
)

plt.xlabel('Episode')
plt.ylabel('Total Reward')

plt.title(
    'Q-Learning Convergence: Grid World (4x4)'
)

plt.legend()
plt.tight_layout()

plt.savefig(
    'imgs/a11_qlearning.png',
    dpi=130
```

```
)
print(
    "\nPlot saved. Agent converges to shortest "
    "obstacle-avoiding path to goal."
)
```

Output:

Training complete over 500 episodes.

Average reward (first 50 episodes) : -2.08

Average reward (last 50 episodes) : 4.86

Learned Optimal Policy (grid view):

> > v v

v X v v

> > > v

> > > G

Plot saved. Agent converges to shortest obstacle-avoiding path to goal.

